

Managing Users, Groups, and Sudo Privileges? SSH Security & Key-Based Authentication? Understanding SELinux & AppArmor

Q.1 Create a new user called streamer and a group called musicfans on your Linux system, then add streamer to the musicfans group. Verify the group membership using the id command.

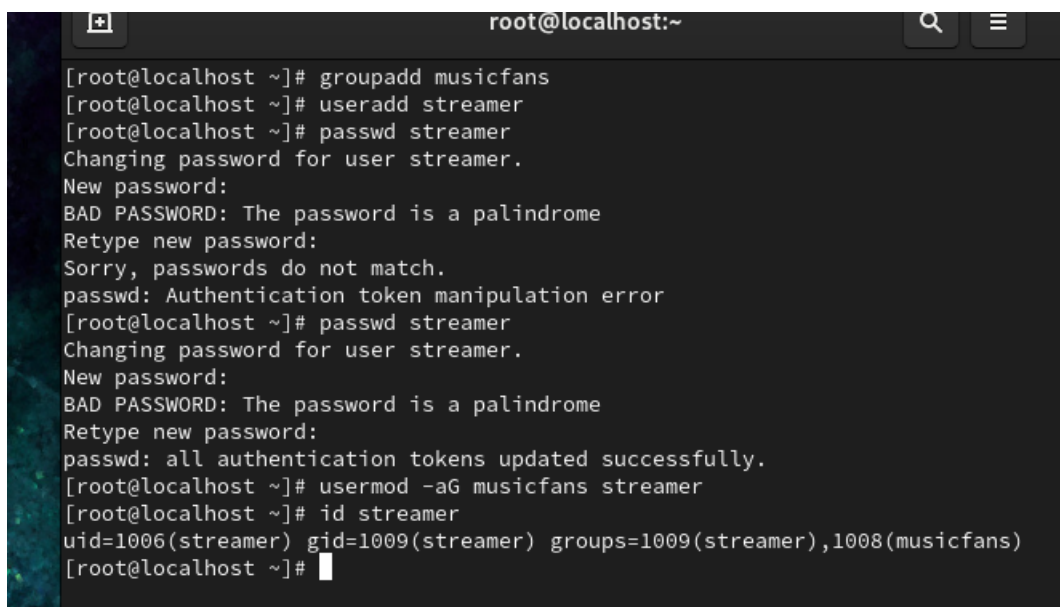
Ans- Step 1- groupadd musicfans

Step 2- useradd streamer

Step 3- passwd streamer

Step 4- usermod -aG musicfans streamer

Step 5- id streamer

A terminal window titled 'root@localhost:~' showing the following commands and output:

```
[root@localhost ~]# groupadd musicfans
[root@localhost ~]# useradd streamer
[root@localhost ~]# passwd streamer
Changing password for user streamer.
New password:
BAD PASSWORD: The password is a palindrome
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
[root@localhost ~]# passwd streamer
Changing password for user streamer.
New password:
BAD PASSWORD: The password is a palindrome
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]# usermod -aG musicfans streamer
[root@localhost ~]# id streamer
uid=1006(streamer) gid=1009(streamer) groups=1009(streamer),1008(musicfans)
[root@localhost ~]#
```

Q.2 Give your current user sudo privileges by editing the /etc/sudoers file using visudo. Test by running a command that requires sudo, such as updating the package list.

Ans- Step 1- visudo

Step 2- root ALL=(ALL)ALL

Step 3- Test: sudo dnf update

```
root@localhost:~ — sudo dnf update

[root@localhost ~]#
[root@localhost ~]# visudo
[root@localhost ~]# sudo dnf check update
usage: dnf check [-c [config file]] [-q] [-v] [--version] [--installroot [path]] [--nodocs] [--noplugins] [--enableplugin [plugin]]
                [--disableplugin [plugin]] [--releasever RELEASEVER] [--setopt SETOPTS] [--skip-broken] [-h] [--allow-erasing] [-b | --nobest] [-C]
                [-R [minutes]] [-d [debug level]] [--debugsolver] [--showduplicates] [-e ERRORLEVEL] [--obsoletes] [--rpmverbosity [debug level name]] [-y]
                [--assumeno] [--enablerepo [repo]] [--disablerepo [repo] | --repo [repo]] [--enable | --disable] [-x [package]] [--disableexcludes [repo]]
                [--repofrompath [repo,path]] [--noautoremove] [--nogpgcheck] [--color COLOR] [--refresh] [-4] [-6] [--destdir DESTDIR] [--downloadonly]
                [--comment COMMENT] [--bugfix] [--enhancement] [--newpackage] [--security] [--advisory ADVISORY] [--bz BUGZILLA] [--cve CVES]
                [--sec-severity {Critical,Important,Moderate,Low}] [--forcearch ARCH] [--all] [--dependencies] [--duplicates] [--obsoleted] [--provides]
dnf check: error: argument check_yum_types: invalid choice: 'update' (choose from 'all', 'dependencies', 'duplicates', 'obsoleted', 'provides', [])
[root@localhost ~]# sudo dnf update
Last metadata expiration check: 0:07:27 ago on Mon 29 Jun 2026 12:39:00 AM EDT.
Dependencies resolved.

=====
Package                                Architecture    Version          Repository        Size
=====
Installing:
kernel                                x86_64          5.14.0-710.el9   baseos            251 k
mesa-compat-libxatracker              x86_64          25.0.7-2.el9     appstream         1.3 M
replacing mesa-libxatracker.x86_64 24.1.2-3.el9
Upgrading:
NetworkManager                       x86_64          1:1.54.4-1.el9   baseos            2.4 M
NetworkManager-adsl                  x86_64          1:1.54.4-1.el9   baseos            30 k
NetworkManager-bluetooth             x86_64          1:1.54.4-1.el9   baseos            56 k
NetworkManager-config-server         noarch          1:1.54.4-1.el9   baseos            16 k
NetworkManager-libnm                 x86_64          1:1.54.4-1.el9   baseos            1.9 M
NetworkManager-team                   x86_64          1:1.54.4-1.el9   baseos            35 k
NetworkManager-tui                    x86_64          1:1.54.4-1.el9   baseos            246 k
NetworkManager-wifi                   x86_64          1:1.54.4-1.el9   baseos            79 k
=====
```

**Q.3 Set up SSH key-based authentication: generate an SSH key pair using ssh-keygen, then copy your public key to a remote Linux server (or a local VM) using ssh-copy-id. Disable password authentication for SSH in /etc/ssh/sshd_config and restart the SSH service.

Hint: Look for the PasswordAuthentication setting in the SSH config file.**

Ans- Step 1-Generate SSH key: ssh-keygen

Step 2- Copy the public key: ssh-copy-id user@server-ip

Step 3- Edit SSH Configuration: vi/etc/ssh/sshd_config

Step 4-Restart SSH: systemctl restart sshd

```
root@localhost:~  
[root@localhost ~]# ssh-keygen  
Generating public/private rsa key pair.  
Enter file in which to save the key (/root/.ssh/id_rsa): ssh-copy-id user@server-ip  
Enter passphrase (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in ssh-copy-id user@server-ip  
Your public key has been saved in ssh-copy-id user@server-ip.pub  
The key fingerprint is:  
SHA256:Y0vhKvF0RMhc/x4NvRBovonEokE0d68/kRt+te9phK4 root@localhost.localdomain  
The key's randomart image is:  
+---[RSA 3072]---+  
| .00.00 .. |  
| .0+..00 0 |  
| . 0000 0 . |  
| . ..++..0 + |  
| o ++.Soo + |  
| o .00+0= 0... |  
| + .0 = 0... |  
| . .. 0 .... |  
| .. E. o+ |  
+---[SHA256]-----+  
[root@localhost ~]# vi /etc/ssh/sshd_config  
[root@localhost ~]# systemctl restart sshd  
[root@localhost ~]#
```

Q.4 Check if SELinux or AppArmor is enabled on your system. If SELinux is active, use the getenforce command to display its current mode, and switch it to permissive mode. If AppArmor is active, list all enforced profiles using sudo apparmor_status.

Ans- Step 1- Check status: getenforce

Step 2- Shows: Enforcing

Step 3- Change to permissive: setenforce 0

Step 4 - Verify: getenforce

```
[root@localhost ~]# getenforce  
Enforcing  
[root@localhost ~]# setenforce 0  
[root@localhost ~]# getenforce  
Permissive  
[root@localhost ~]#
```

Q.5 Use ChatGPT to explain the difference between SELinux and AppArmor in one paragraph, then write a summary in your own words and mention one real-world app (like Zomato or Paytm) that would benefit from these security modules.

Ans- SELinux and AppArmor are linux security modules that provide mandatory access control to protect the sytem. SELinux uses security labels and offers very detailed and strict security polociies,making it highly secure but more complex to configure. AppArmor uses file paths to define permissions, making it simpler to manage but lessgranular than SELinux. Both help prevent unauthorized access and limit the damage if an applicatiob is compromised.